



Faculty of Engineering
Cairo University



Cairo University

EECS316 Communications 2

Matched Filter Assignment

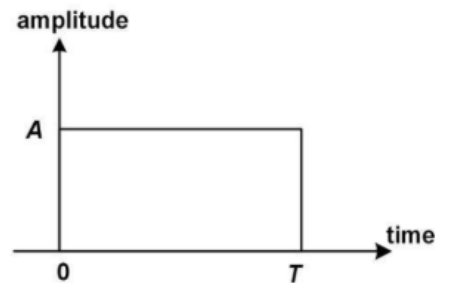
Team 14

Name	ID
Abdullah Haytham Mohamed	1210253
Basma Mohamed Abd El Hakeem	1210208
Danah Rafik Reyad Aly Hassan	1210221

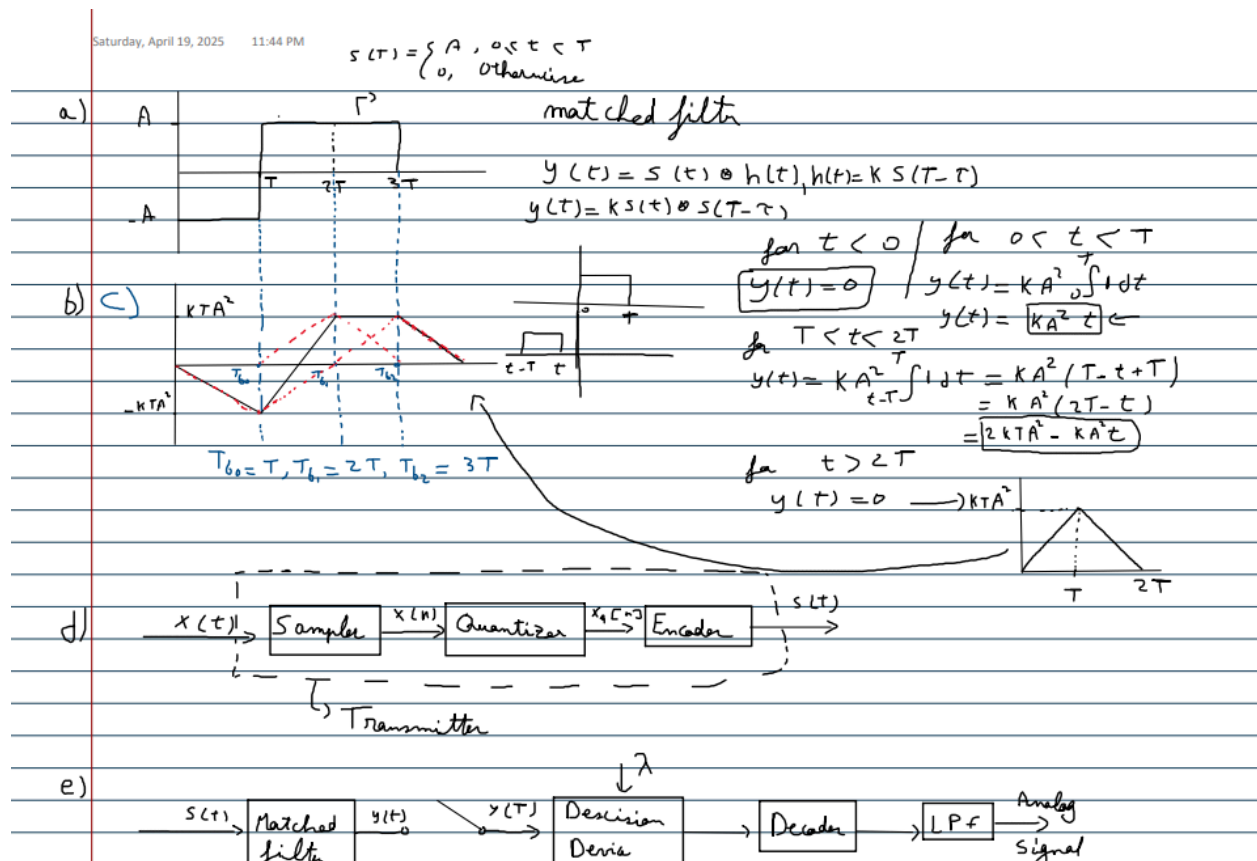
Part I: Solve the following question:

Given the pulse shape in Fig.1 and assuming that '1' and '0' are represented by a positive and a negative pulse, respectively.

- Plot the transmitted baseband waveform $s(t)$ for the bit sequence $b_0 = '0'$, $b_1 = '1'$ and $b_2 = '1'$
- Plot the matched filter output due to signal only, i.e., ignore the noise
- Mark the sampling instants to detect b_0 , b_1 and b_2
- Plot the block diagram of the transmitter
- Plot the block diagram of the receiver



Hand analysis:



Code using Octave:

```
% Parameters
A = 5;           % Amplitude
T = 1;           % Bit duration
k = 1;           % Constant for matched filter
bit_seq = [0 1 1];
% Time resolution
dt = 0.001;
t = 0:dt:(length(bit_seq)*T - dt);
% Generate original signal
signal = zeros(size(t));
for i = 1:length(bit_seq)
    idx = (t >= (i-1)*T) & (t < i*T);
    if bit_seq(i) == 1
        signal(idx) = A;
    else
        signal(idx) = -A;
    end
end
% Create matched filter  $h(t) = k * S(T - t)$ 
t_filter = 0:dt:T-dt;
pulse = A * ones(size(t_filter));
matched_filter = k * fliplr(pulse);

% Convolution (matched filtering)
filtered_output = conv(signal, matched_filter) * dt;
t_conv = 0:dt:(length(filtered_output)-1)*dt;

% Sampling times (at end of each bit)
sampling_times = T:T:(length(bit_seq)*T);
[~, sample_indices] = min(abs(t_conv' - sampling_times), [], 1);
sample_values = filtered_output(sample_indices);

% Plotting
figure;

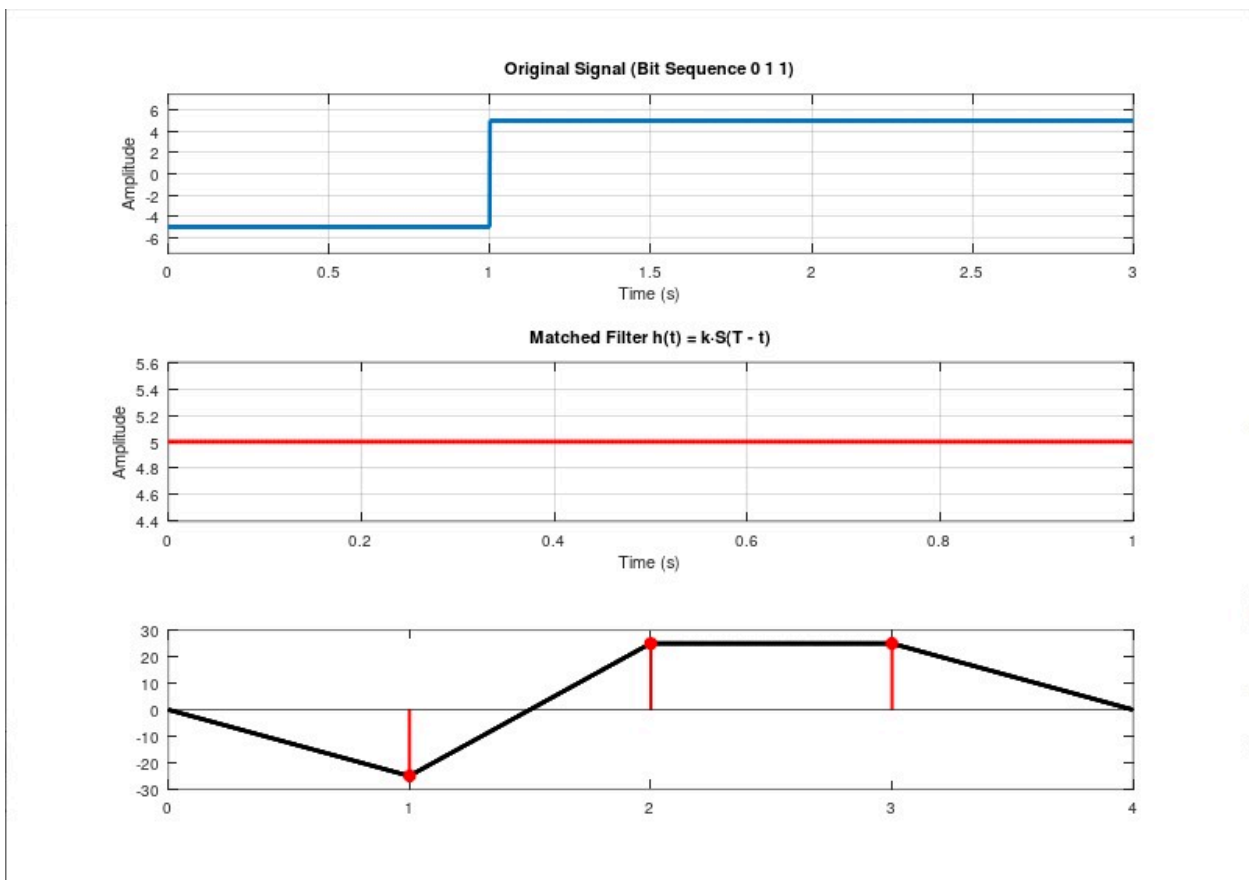
subplot(3,1,1);
plot(t, signal, 'LineWidth', 2);
title('Original Signal (Bit Sequence 0 1 1)');
xlabel('Time (s)');
ylabel('Amplitude');
ylim([-A*1.5, A*1.5]);
grid on;

subplot(3,1,2);
```

```

plot(t_filter, matched_filter, 'r', 'LineWidth', 2);
title('Matched Filter  $h(t) = k \cdot S(T - t)$ ');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
subplot(3,1,3);
plot(t_conv, filtered_output, 'k', 'LineWidth', 2); hold on;
stem(sampling_times, sample_values, 'r', 'filled', 'LineWidth', 1.5);
for i = 1:length(sampling_times)
    xline(sampling_times(i), '--b');
end
title('Matched Filter Output with Sampling Times');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Convolution Output', 'Sampled Values', 'Sampling Times');
grid on;

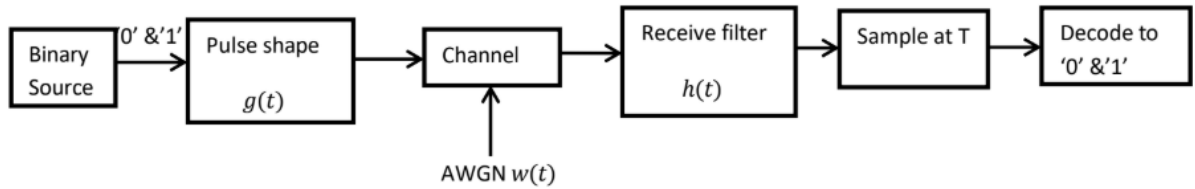
```



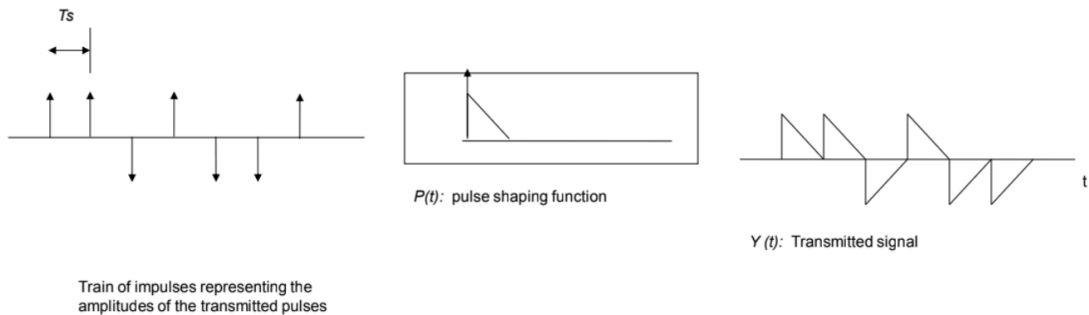
The above figure shows the convolution between the bit sequence 011 and the matched filter. It produces triangle-like pulses for each of the rectangular pulses.

Part II: Simulation:

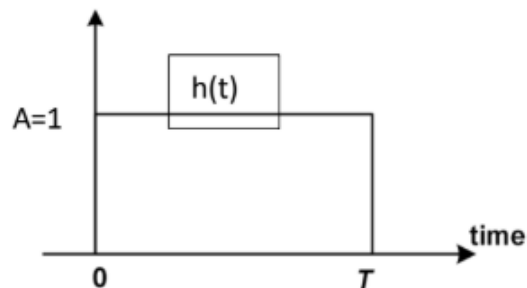
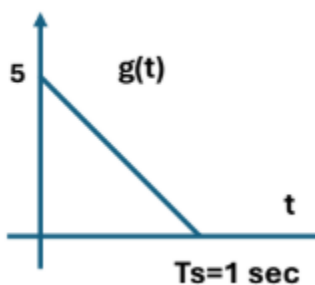
Consider the following communication system.



- The output of the binary source is a series of random 0's and 1's.
- The after the pulse shaping $g(t)$ where '1' is represented by $g(t)$ and '0' by $-g(t)$



- The pulse shape $g(t)$ is given below has $T = 1$ sec:
- The channel is ideal (i.e. its impulse response is $\delta(t)$).
- The noise is an AWGN with zero mean and variance $N_0/2$.
- Consider the three following cases:
 - a) The receive filter $h(t)$ is a matched filter with unit energy
 - b) The receive filter $h(t)$ is not existent (i.e. $h(t) = \delta(t)$)
 - c) The receive filter $h(t)$ has the following impulse response

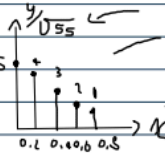
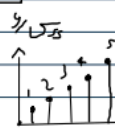


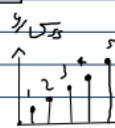
Part II Requirements:

1. Derive the probability of error in the three mentioned cases.

Hand analysis:

Problem 2

1) $g(t) = s$  $g(t) = \frac{[5 \ 4 \ 3 \ 2 \ 1]}{\sqrt{55}}$
 $E_g = \sum_{n=0}^N g(n)^2 = 1$  $\rightarrow E_h = 1 \rightarrow \text{given}$

2) for $h(t) = \text{matched filter of } g(t) \rightarrow h(t) =$  $\rightarrow E_h = 1 \rightarrow \text{given}$
 $y_i(n) = g(n) \otimes h(n) = \sum_{k=0}^N g(k) h(i-n) = \sum_{k=0}^N |g(k)|^2 = E_g = 1 \Rightarrow S_{11} = E_g$
 $\sigma_y^2 = \sigma_n^2 E_h = \frac{N_0}{2} \cdot 1 \rightarrow N = N_0$
 $S_{21} = -S_{11} = -1$

$\therefore$ bit generation random

$\therefore P(0) = P(1)$ and $\lambda = \frac{S_{11} + S_{21}}{2} + \frac{N/2}{S_{11} - S_{21}} \ln \left(\frac{P_1}{P_0} \right) = 0$

$\therefore \lambda = 0$ and $P(0) = P(1)$

$\therefore \text{BER} = Q \left(\frac{S_{11} - S_{21}}{\sqrt{2N}} \right) = Q \left(\frac{2\sqrt{E_g}}{\sqrt{2N}} \right) = Q \left(\sqrt{\frac{2E_g}{N_0}} \right) = Q \left(\sqrt{\frac{2}{N_0}} \right)$

3) for $h(t) = \delta(t)$,

$E_g = \sum |g(n)|^2 = 1$, $S_{11} = \frac{1}{\sqrt{55}} \rightarrow S_{21} = -S_{11} = -\frac{1}{\sqrt{55}}$

$\phi(n) = \frac{g(n)}{\sqrt{E_g}} \rightarrow \phi(n) = \frac{g(n)}{1} = g(n)$

$\lambda = 0$ (Same as A), $\text{BER} = Q \left(\frac{S_{11} - S_{21}}{\sqrt{2N_0}} \right) = Q \left(\frac{2/\sqrt{55}}{\sqrt{2N_0}} \right) = Q \left(\sqrt{\frac{2E_g}{55N_0}} \right)$

4) for $h(n) =$  $\rightarrow$ to normalize it $\rightarrow E = 1 = \sum_{n=0}^N A^2 = 5A^2 = 1$
 $A = \frac{1}{\sqrt{5}}$

$y_3(n) = g(n) \otimes h(n) = \sum g(n) h(i-n) = \sum g(n) h(n) = \frac{1}{\sqrt{5}} (5+4+3+2+1) = 3\sqrt{11}$

$E_h = 1 \rightarrow \sigma_y^2 = \sigma_n^2 E_h = \frac{N_0}{2} E_h \rightarrow N = N_0$
 $S_{11} = \frac{3\sqrt{11}}{11}$, $S_{21} = -S_{11} = -\frac{3\sqrt{11}}{11}$

$\lambda \rightarrow$ Same as A, $A = 0$

$\text{BER} = Q \left(\frac{S_{11} - S_{21}}{\sqrt{2N}} \right) = Q \left(\frac{3}{11} \sqrt{\frac{2 \times 11 E_g}{N_0}} \right) = Q \left(\frac{3}{11} \sqrt{\frac{22 E_g}{N_0}} \right)$

5) BER is decreasing function with $\frac{E}{N_0}$ because $\frac{E}{N_0} \propto \text{SNR}$ and when signal to noise ratio increases the BER decreases

6) Case 1 has the lowest BER because matched filter maximizes SNR so it has lowest error

2. Write a Matlab code that generates random bits, simulates the above communication system, and calculates the probability of error for the three mentioned cases.

Code using Octave:

```
% Clear environment
clc; clear; close all;
pkg load control
pkg load communications
pkg load image

% Custom upsample function
function y = upsample_manual(x, n)
    y = zeros(1, n * length(x));
    y(1:n:end) = x;
end

% Define the signal g(n)
g = [5 4 3 2 1] / sqrt(55);

% Define filters
h_matched = flip1r(g); % Matched filter
h_impulse = [1]; % Impulse response
h_rect = ones(1, 5) / sqrt(5); % Rect filter with unity energy

% BER Calculation with 100,000 bits
N = 100000;
Ts = 5;
num_bits = ceil(N / Ts);
bits = 2 * randi([0 1], 1, num_bits) - 1; % This transforms 0's and 1's into -1's
and 1's

signal = upsample_manual(bits, Ts);
signal = signal(1:N); % Trim to 100k samples

% Transmit through channel (convolve with g)
y_clean = conv(signal, g);

% Add AWGN with SNR = 0 dB
y_noisy = awgn(y_clean, 0, 'measured');

% Function to compute BER
function ber = compute_BER(y_noisy, filter, bits, Ts)
    y_filtered = conv(y_noisy, filter); % Filter y by convolving with filter
```

```

    start_index = length(filter); % Start index is the delay introduced by the
filter to start taking the first sample
    sampling_indices = start_index:Ts:length(y_filtered); % In this case Ts = 5 so
we start at the delay then take sample every 5 bits

    % Ensure we don't access out-of-bounds bits
    max_valid_samples = min(length(sampling_indices), length(bits));
    sampling_indices = sampling_indices(1:max_valid_samples);

    samples = y_filtered(sampling_indices); % Take samples every sampling index
    original_bits = bits(1:max_valid_samples);
    received_bits = samples > 0; % Transforms array to 0's and 1's again (Logical
array)

    num_errors = sum(received_bits ~= (original_bits > 0)); % XOR's the original
bits with received bits and counts the 1's (differences)
    ber = num_errors / max_valid_samples; % Num of errors / Total num of bits
end

% Calculate BERs
ber_matched = compute_BER(y_noisy, h_matched, bits, Ts);
ber_impulse = compute_BER(y_noisy, h_impulse, bits, Ts);
ber_rect = compute_BER(y_noisy, h_rect, bits, Ts);

% Print Results
fprintf("BER with matched filter: %.4f\n", ber_matched);
fprintf("BER with impulse filter: %.4f\n", ber_impulse);
fprintf("BER with rect filter    : %.4f\n", ber_rect);

```

```

BER with matched filter: 0.0127
BER with impulse filter: 0.0664
BER with rect filter    : 0.0219

```

The above print statements show that the matched filter has the lowest BER while the impulse has the highest BER

3. Plot the output of the receive filter for the three mentioned cases for three bit only.

```

% 3-bit example for visualization

```



```

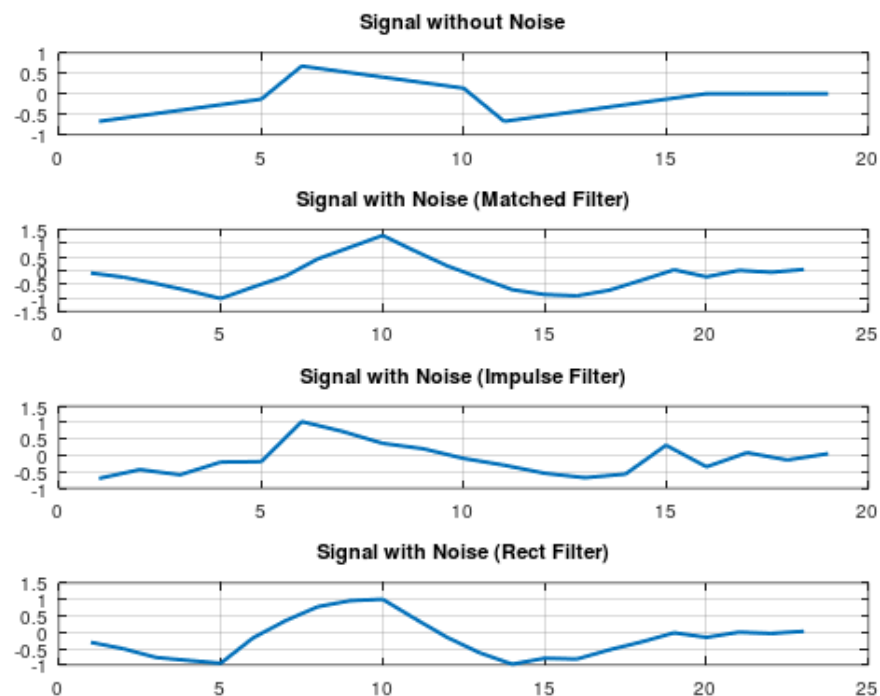
bits3 = 2 * randi([0 1], 1, 3) - 1; % Creates 3 random bits that are either 1 or -1
signal3 = upsample_manual(bits3, 5);
signal3 = signal3(1:length(bits3) * 5);

y_clean3 = conv(signal3, g); % Convolve the g with the 3 bits to get a train of g's
(+ve and -ve)
y_noisy3 = awgn(y_clean3, 5, 'measured'); % Adds Noise with SNR = 5dB

% Passes noisy signal through each filter
y_noisy_matched = conv(y_noisy3, h_matched);
y_noisy_impulse = conv(y_noisy3, h_impulse);
y_noisy_rect = conv(y_noisy3, h_rect);

% Plotting
figure;
subplot(4,1,1);
plot(y_clean3, 'LineWidth', 1.5); grid on;
title('Signal without Noise');
subplot(4,1,2);
plot(y_noisy_matched, 'LineWidth', 1.5); grid on;
title('Signal with Noise (Matched Filter)');
subplot(4,1,3);
plot(y_noisy_impulse, 'LineWidth', 1.5); grid on;
title('Signal with Noise (Impulse Filter)');
subplot(4,1,4);
plot(y_noisy_rect, 'LineWidth', 1.5); grid on;
title('Signal with Noise (Rect Filter)');

```



The above figure shows that at the sampling points the matched filter is almost always in a maximum or a minimum, unlike the impulse filter where the noise has more effect on it. The rect filter shows similar results to the matched filter.

4. On the same figure, plot the theoretical and simulated Bit Error Rate (BER) Vs E/No (where E is the average symbol energy) for the three mentioned cases. Take E/No to be in the range -10dB: 1 : 10dB. (Use a semilogy plot).

```
% Theoretical vs Simulated BER Plot
SNRs = -10:1:10;
% Creates arrays with size of SNRs array (to store value of each filter for every SNR)
sim_ber_matched = zeros(1, length(SNRs));
sim_ber_impulse = zeros(1, length(SNRs));
sim_ber_rect = zeros(1, length(SNRs));
% Compute BERs for different SNR values
for i = 1:length(SNRs)
    y_noisy_loop = awgn(y_clean, SNRs(i), 'measured'); % Adds Additive White Gaussian Noise to clean signal
    % Calculates the experimental BER for each array
    sim_ber_matched(i) = compute_BER(y_noisy_loop, h_matched, bits, Ts);
    sim_ber_impulse(i) = compute_BER(y_noisy_loop, h_impulse, bits, Ts);
    sim_ber_rect(i) = compute_BER(y_noisy_loop, h_rect, bits, Ts);
end
% Theoretical BERs
BER_theo_A = qfunc(sqrt(2 .* 10.^(SNRs / 10))); % Matched
BER_theo_B = qfunc(sqrt(2 / 55) .* 10.^(SNRs / 10)); % Impulse
BER_theo_C = qfunc((3 / 11) .* sqrt(22 .* 10.^(SNRs / 10))); % Rect
% Plot for Matched Filter
figure;
semilogy(SNRs, sim_ber_matched, 'o-', 'LineWidth', 1.5); hold on;
semilogy(SNRs, BER_theo_A, '--', 'LineWidth', 1.5);
xlabel('E/No (dB)');
ylabel('Bit Error Rate (BER)');
title('Theoretical vs Simulated BER for Matched Filter');
legend({'Simulated BER', 'Theoretical BER'}, 'Location', 'SouthWest');
grid on;
% Plot for Impulse Filter
figure;
semilogy(SNRs, sim_ber_impulse, 's-', 'LineWidth', 1.5); hold on;
semilogy(SNRs, BER_theo_B, '--', 'LineWidth', 1.5);
xlabel('E/No (dB)');
ylabel('Bit Error Rate (BER)');
title('Theoretical vs Simulated BER for Impulse Filter');
legend({'Simulated BER', 'Theoretical BER'}, 'Location', 'SouthWest');
```

```
grid on;
```

```
% Plot for Rect Filter
```

```
figure;
```

```
semilogy(SNRs, sim_ber_rect, 'd-', 'LineWidth', 1.5); hold on;
```

```
semilogy(SNRs, BER_theo_C, '--', 'LineWidth', 1.5);
```

```
xlabel('E/No (dB)');
```

```
ylabel('Bit Error Rate (BER)');
```

```
title('Theoretical vs Simulated BER for Rect Filter');
```

```
legend({'Simulated BER', 'Theoretical BER'}, 'Location', 'SouthWest');
```

```
grid on;
```

```
% Combined Plot for All Filters
```

```
figure;
```

```
semilogy(SNRs, sim_ber_matched, 'o-', 'LineWidth', 1.5); hold on;
```

```
semilogy(SNRs, sim_ber_impulse, 's-', 'LineWidth', 1.5);
```

```
semilogy(SNRs, sim_ber_rect, 'd-', 'LineWidth', 1.5);
```

```
semilogy(SNRs, BER_theo_A, '--', 'LineWidth', 1.5);
```

```
semilogy(SNRs, BER_theo_B, '--', 'LineWidth', 1.5);
```

```
semilogy(SNRs, BER_theo_C, '--', 'LineWidth', 1.5);
```

```
xlabel('E/No (dB)');
```

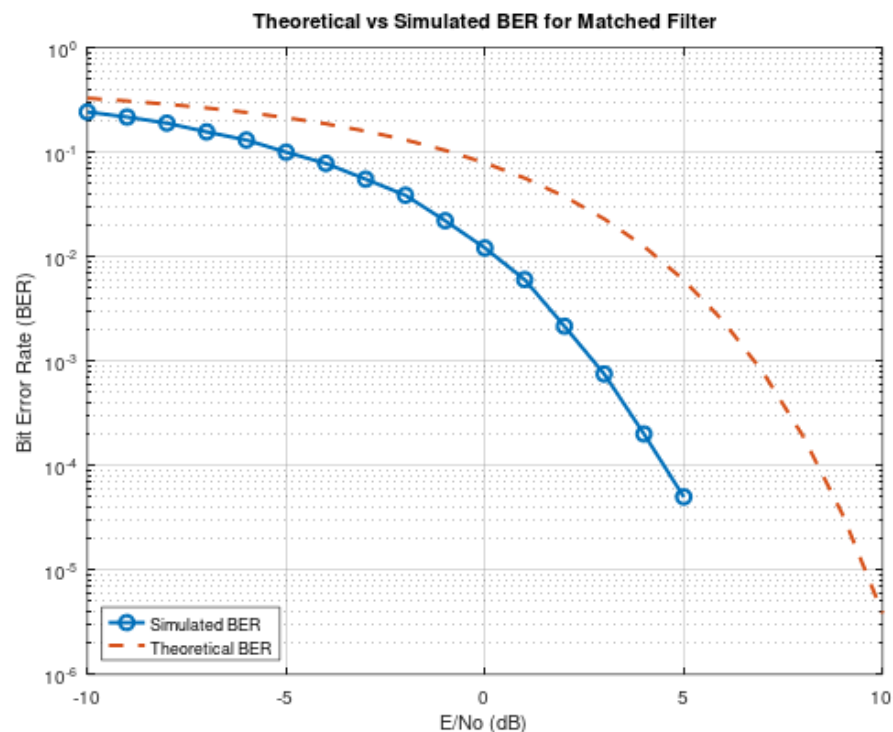
```
ylabel('Bit Error Rate (BER)');
```

```
title('Theoretical vs Simulated BER for All Filters');
```

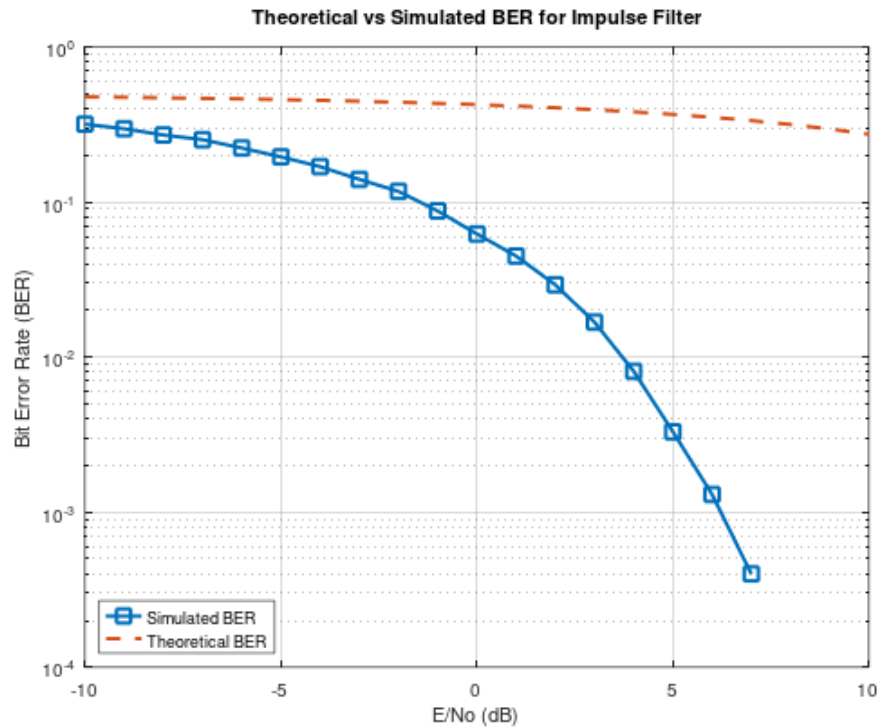
```
legend({'Simulated Matched', 'Simulated Impulse', 'Simulated Rect', ...  
        'Theoretical Matched', 'Theoretical Impulse', 'Theoretical Rect'}, ...  
        'Location', 'SouthWest');
```

```
grid on;
```

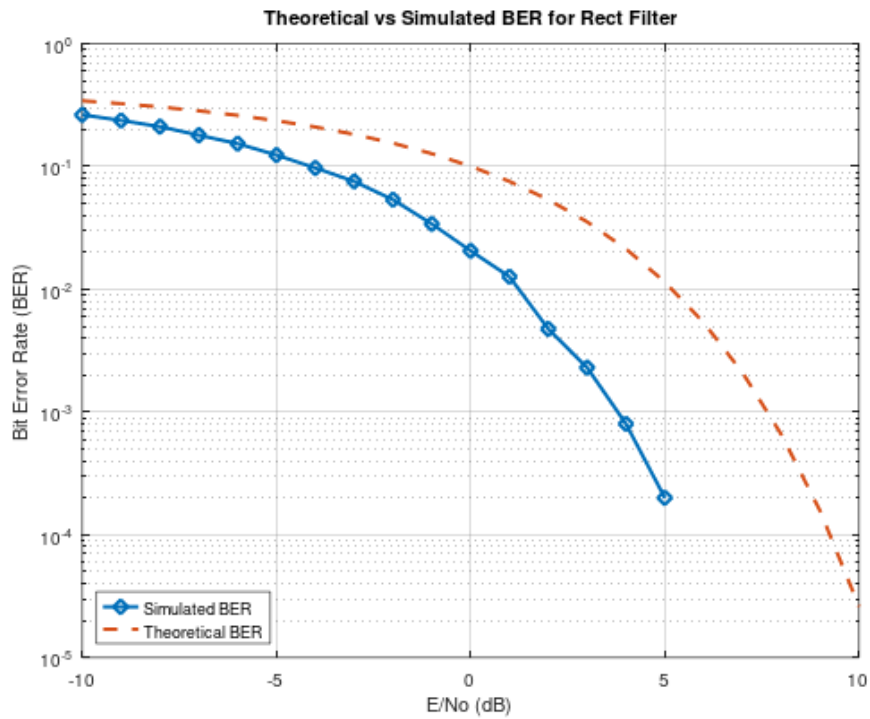
a- Figure for each filter contains theoretical and simulated Bit Error Rate (BER)



For the matched filter the simulated values are really close to the theoretical as we can see from the graph

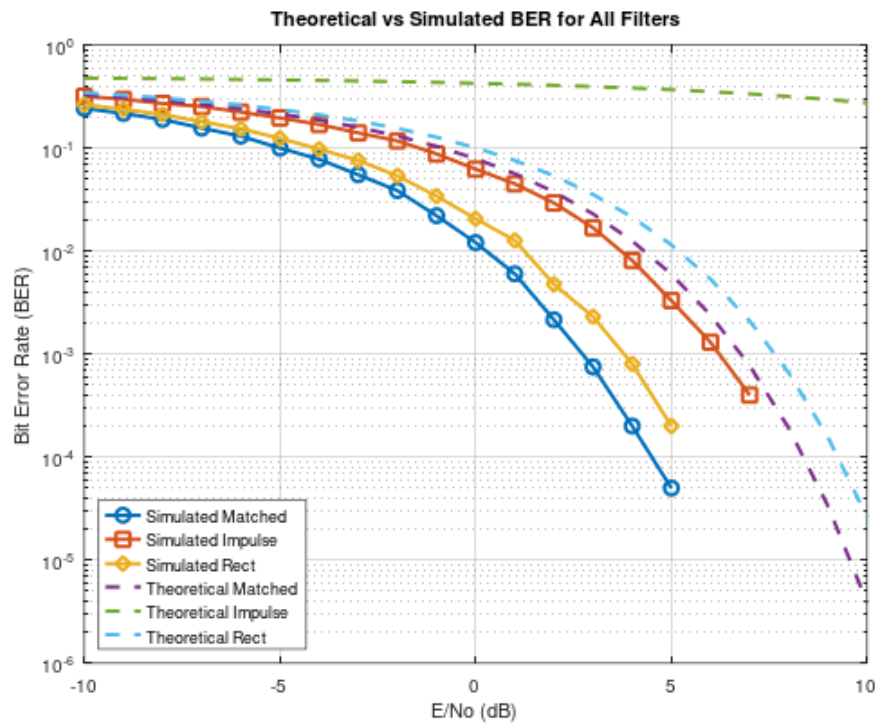


For the impulse filter the theoretical shows the error to be much higher than the simulated value and I suspect this is because we needed more samples to more accurately match it



For the rect filter the simulated values are really close to the theoretical as we can see from the graph

b- Figure contains all three cases together.



From the combined graph we can see that the matched filter has the lowest theoretical and simulated BER out of all them followed by the rect filter and the impulse filter had the highest simulated and theoretical values

5. Is the BER an increasing or a decreasing function of E/No? Why?

BER is a decreasing function with E/No because E/No is proportional to the SNR and when the SNR increases the BER decreases

6. Which case has the lowest BER? Why?

Case 1 has the lowest BER because the matched filter maximizes the SNR and therefore it has the lowest error

7. Send Three Images as your bits stream through your system with SNR = -10, -5, 0, 5;

```
% Define input images and SNR values
image_files = {'image1.png', 'image2.png', 'image3.png'};
SNRs = [-10, -5, 0, 5];
target_size = [200, 200]; % Resize all images to this size

% Pulse shaping parameters
delay = length(h_matched) - 1;

for snr_idx = 1:length(SNRs)
    SNR = SNRs(snr_idx); % Current SNR
    fprintf('Processing for SNR = %d dB\n', SNR);

    figure;

    for i = 1:length(image_files)
        % Load image from file
        img = imread(image_files{i});

        % If the image is grayscale (2D), replicate channels to convert to RGB (3D)
        if size(img, 3) == 1
            img = cat(3, img, img, img); % Convert grayscale to RGB
        end

        img = imresize(img, target_size); % Resize the image to the target size to make processing easier

        received_img = zeros(size(img), 'uint8'); % We will store received img here (uint8 because we store 8 bits
```

because each pixel is 8 bits)

```
% Process each of the 3 RGB color channels separately
for channel = 1:3
    channel_data = img(:, :, channel); % Extract current channel

    bits = reshape(de2bi(channel_data(:, 8, 'left-msb')), 1, []); % Convert pixel values (0-255) to binary
    % bitstream (8 bits per pixel)

    pam_signal = 2 * bits - 1; % Map binary bits {0,1} to PAM symbols {-1, +1}

    y_img_clean = conv(upsample_manual(pam_signal, 8), g); % Upsample and pulse shape using
    % convolution with transmit filter `g`

    y_img_noisy = awgn(y_img_clean, SNR, 'measured'); % Add white Gaussian noise to the clean signal

    filtered_signal = conv(y_img_noisy, h_matched); % Apply matched filter

    % Sample the filtered signal at symbol intervals (with delay)
    sample_indices = delay + 1 : 8 : delay + length(bits) * 8;
    sampled_bits = filtered_signal(sample_indices) > 0; % Transforms array to 0's and 1's again (logical
    % array)

    % Reshape bitstream back into 8-bit integers (bytes)
    reshaped_bits = reshape(sampled_bits, 8, []);
    decoded_vals = bi2de(reshaped_bits, 'left-msb'); % Convert bits to pixel values

    % Reshape flat vector back to 2D image matrix
    channel_received = reshape(decoded_vals, target_size);

    % Store received channel into the final image
    received_img(:, :, channel) = uint8(channel_received);
end

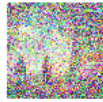
% Display the original image in a subplot
subplot(length(image_files), 2, (i - 1)*2 + 1);
imshow(img);
title(sprintf('Original Image %d', i));

% Display the received (noisy and decoded) image in a subplot
subplot(length(image_files), 2, (i - 1)*2 + 2);
imshow(received_img);
title(sprintf('Received Image %d (SNR: %d dB)', i, SNR));
end
end
```

Original Image 1



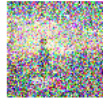
Received Image 1 (SNR: -10 dB)



Original Image 2



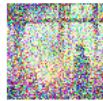
Received Image 2 (SNR: -10 dB)



Original Image 3



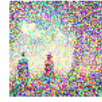
Received Image 3 (SNR: -10 dB)



Original Image 1



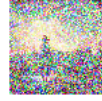
Received Image 1 (SNR: -5 dB)



Original Image 2



Received Image 2 (SNR: -5 dB)



Original Image 3



Received Image 3 (SNR: -5 dB)



Original Image 1



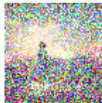
Received Image 1 (SNR: 0 dB)



Original Image 2



Received Image 2 (SNR: 0 dB)



Original Image 3



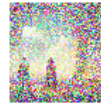
Received Image 3 (SNR: 0 dB)



Original Image 1



Received Image 1 (SNR: 5 dB)



Original Image 2



Received Image 2 (SNR: 5 dB)



Original Image 3



Received Image 3 (SNR: 5 dB)



From the images above we can see that when the SNR is lowest (-10dB) the effect of the noise on the image is biggest even after filtering it and when the SNR is highest (5dB) the images are slightly clearer